

DeepSeek Harness vs OpenCode

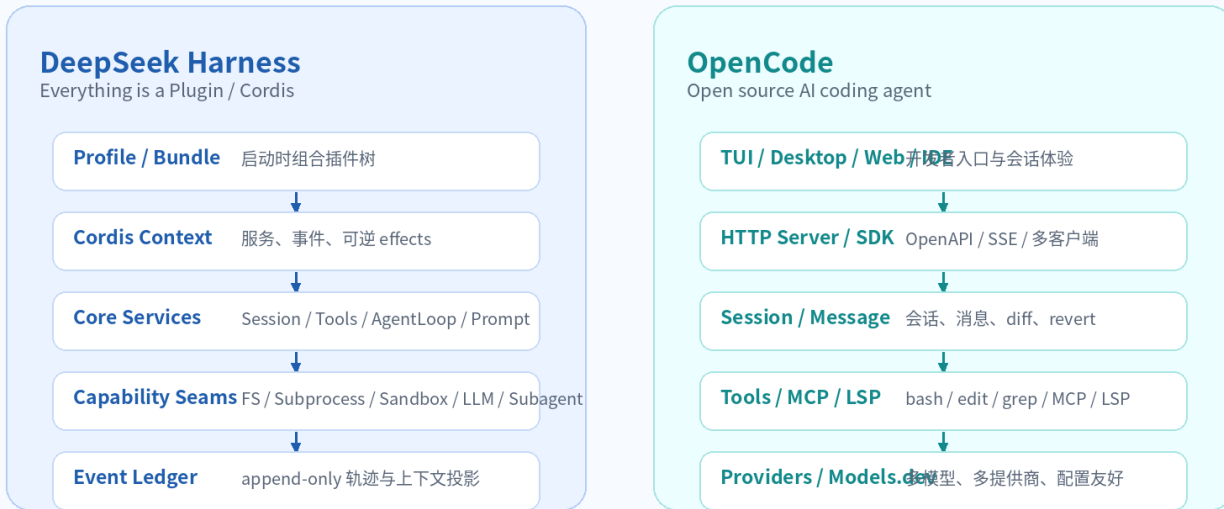
综合比较分析报告

设计原理 / 源码结构 / 设计模式 / 使用体验 / 企业落地

调研日期：2026-08-18 | 范围：DeepSeek 官方 deepseek-ai/deepseek-harness 与 OpenCode
anomalyco/opencode

架构定位对比：运行时内核 vs 产品化编码代理

DeepSeek Harness 更像可替换的 Agent OS Kernel；OpenCode 更像面向开发者的完整 coding agent 产品。



核心判断：DSH 的价值在“可治理内核”；OpenCode 的价值在“好用、易部署、入口完整”。

图 1：两者的总体定位差异

一句话结论：OpenCode 更像面向开发者的成熟 Coding Agent 产品，DeepSeek Harness 更像一个正在快速演化的 Agent Runtime / Agent OS 内核。前者在安装、TUI/桌面/服务端、多模型和 MCP 生态上更适合直接落地；后者在插件树、capability seam、append-only 轨迹、ToolRuntime、PTC/Code Mode、sandbox fail-closed 等底层设计上更值得企业平台建设借鉴。

0. 阅读说明与调研边界

本报告基于官方仓库、官方文档与关键源码文件进行静态调研。未在同一任务集上对两者进行统一 benchmark，也未统计真实用户时延、成功率或成本。因此，本文的“体验”部分主要来自产品形态、配置入口、API/CLI/界面能力与源码可验证的执行路径，而不是实验跑分。

- DeepSeek Harness 当前官方 README 明确处于 developer preview，并提示会出现 compatibility-breaking changes。报告中对其架构评价更偏“可借鉴设计”，不是稳定性背书。
- OpenCode 官方 README 与文档将其定位为 open source AI coding agent，并提供 TUI、Desktop、Server、SDK、MCP、Providers 等用户入口；报告会把它作为更成熟的开发者产品来比较。
- OpenCode 的 `packages/llm/DESIGN.md` 是设计草案/迁移意图，不等于全部已落地实现；报告中会标注为“设计方向”。
- 两者都在快速迭代。建议把本报告视为 2026-08 当前快照，而不是永久结论。

1. 总览结论

问题	更推荐	核心理由
今天给团队成员直接使用、替代部分 Claude Code / Cursor 命令行编码 workflow	OpenCode	安装路径成熟，TUI/桌面/服务端/多模型/MCP/权限配置完整，用户路径更短。
建设公司内部 AI Harness / Agent 平台内核	DeepSeek Harness	Cordis 插件树、capability seam、append-only session log、ToolRuntime pipeline、PTC/Code Mode 更像平台内核。
需要长期积累可训练、可回放、可审计的 Agent 轨迹	DeepSeek Harness	SessionEvent log 是 source of truth，deriveMessages 只是投影，天然区分 reality 与 model context。
要快速接入各类 MCP Server 与多模型 provider	OpenCode	OpenCode 的 MCP 配置、server API、provider 生态更贴近实际使用。
要研究 PTC / Programmatic Tool Calling 的底层形态	DeepSeek Harness	DSH 的 Code Mode 是 ToolRuntime presentation；OpenCode 的 Code Mode 更偏 connected MCP tools 的脚本编排。
安全隔离 / 文件写入边界要严格 fail-closed	DeepSeek Harness	read-only / workspace-write 必须有可用 sandbox backend；不可静默退化为裸执行。
低学习成本与可见产品体验	OpenCode	TUI/桌面/Server/API/IDE 文档明确，更容易让普通开发者上手。

如果从企业内部 AI 集成视角看，最现实的结论不是二选一，而是“两层选型”：产品入口可以优先采用 OpenCode 这一类用户体验成熟的 Coding Agent；平台内核、工具契约、审计轨迹、PTC、sandbox、schema 与 policy 则强烈建议吸收 DeepSeek Harness 的设计。

1.1 核心差异地图

维度	DeepSeek Harness	OpenCode	判断
产品定位	开发者预览阶段的 Agent Harness / Runtime Kernel	面向用户的开源 AI Coding Agent	OpenCode 产品化更强，DSH 内核设计更强
架构核心	Cordis 插件树，一切都是 plugin	CLI/TUI/Desktop/Server +	DSH 更偏可替换内核，

		session/tool/provider 层	OpenCode 更偏应用产品
扩展方式	profile/bundle/patch/ capability seam	MCP/custom tools/plugins/OpenAPI/SDK	OpenCode 更易扩展接入，DSH 更系统
Agent Loop	turn/step/event-ledger 驱动	session/message/processor + provider/tool loop	DSH 更强调持久化轨迹， OpenCode 更强调交互和产品
Tool Runtime	统一 pipeline: pre -> guard -> execute -> post -> finalize	工具定义 + Permission allow/deny/ask + MCP	DSH 更像 policy kernel
PTC/Code Mode	native/code/both presentation over ToolRuntime	execute 工具编排 MCP tools	DSH 集成深度更高
Schema	输入+强制输出 schema， canonical value 与 render 分离	Effect Schema + OpenAPI/MCP schemas	DSH 的 Tool Contract 更严格
Sandbox	filesystem-effect sandbox seam, fail-closed	主要是 permission rules，非等 价 OS sandbox	DSH 安全边界更明确
UX	Web/headless, developer preview	TUI、Desktop、Server、IDE、 分享	OpenCode 明显更适合日常使用

1.2 一句话选型

- 1. 要“马上好用”：选 OpenCode。
- 2. 要“研究下一代公司级 Agent Runtime”：读 DeepSeek Harness。
- 3. 要“公司内部可控落地”：OpenCode 做入口，借鉴 DeepSeek Harness 重构工具、轨迹、安全和 PTC 内核。

2. 项目定位与成熟度

2.1 DeepSeek Harness 的定位

DeepSeek Harness 官方 README 将其定义为 DeepSeek AI 开源的 agent harness，并明确其架构是 Everything is a Plugin，底层由 Cordis 驱动。官方同时声明项目仍处于 developer preview，并且会发生破坏兼容的变更。

从源码组织看，它不是只做一个 TUI 或一个 CLI，而是通过 profile/bundle/patch 构造 Cordis 插件树；模型适配器、Tool Registry、Session Log、Agent Loop、Sandbox、Persistence、Web App 等都作为可替换插件存在。

2.2 OpenCode 的定位

OpenCode README 将它描述为 open source AI coding agent，围绕终端、桌面、服务端和 IDE 场景提供完整用户入口。官方 README 已经给出安装脚本、npm/brew/scoop/choco/nix 等路径，并提供 Desktop App beta；文档中还详细介绍了 agents、tools、permissions、MCP、server、SDK 等使用层能力。

OpenCode 的产品路线更接近“面向开发者的可用编码代理”。它既有 TUI 交互，也有 headless server 和 OpenAPI/SSE，适合接入现有开发工具链和自动化流程。

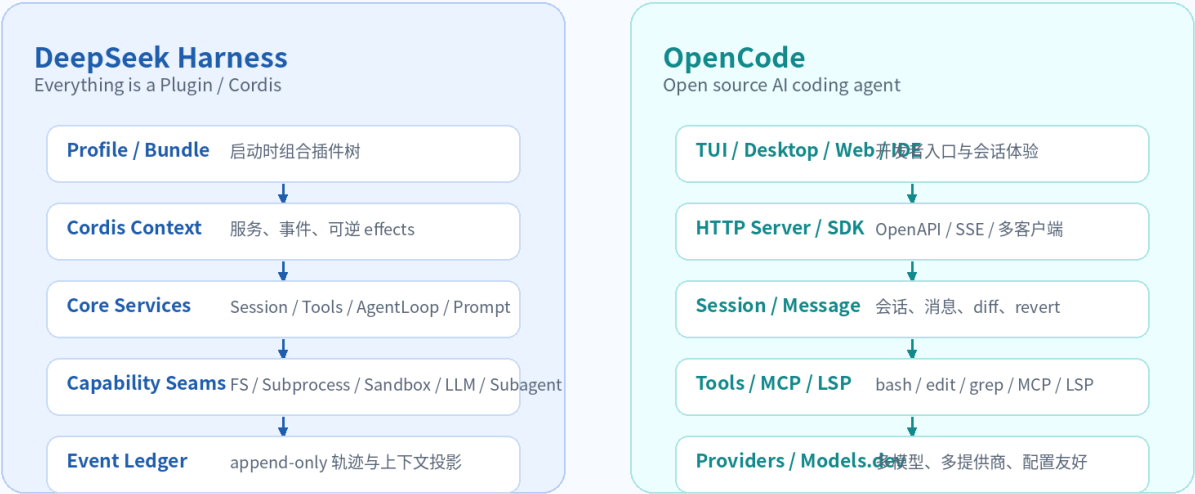
2.3 成熟度判断

方面	DeepSeek Harness	OpenCode
官方状态	developer preview，破坏性变更预期高	面向用户发布，安装与文档更完整
用户入口	Web UI / headless / source build	TUI / Desktop / Server / SDK / IDE / Web docs
平台抽象	极强，几乎所有能力都是 plugin / seam	较强，但更服务于产品运行
源码复杂度	高，学习曲线陡	高，但功能入口更直观
企业落地风险	API 变动与 sandbox 环境适配风险	权限不是 sandbox，MCP/模型配置治理风险

3. 架构原理对比

架构定位对比：运行时内核 vs 产品化编码代理

DeepSeek Harness 更像可替换的 Agent OS Kernel；OpenCode 更像面向开发者的完整 coding agent 产品。



核心判断：DSH 的价值在“可治理内核”；OpenCode 的价值在“好用、易部署、入口完整”。

图 2：DeepSeek Harness 和 OpenCode 的总体架构定位

3.1 DeepSeek Harness : Cordis 插件树 + capability seam

DSH 的架构文档明确说明：Cordis 下的插件贡献 services、typed events 和 reversible effects 到共享 context。模型 adapter、tool registry、session log、agent loop 本身都属于插件，因此理论上都可通过配置替换，而不是在一个 privileged core 中硬改。

它的 profile/bundle 机制让一个运行中的 dsh 从有序层组合而来：base bundle 提供 model adapters、tools、persistence、sandbox、approval policy、settings、credentials、telemetry；web-app 增加浏览器应用；headless 增加无服务器的一次性 runner。

这类架构的优势是：能力边界很清晰，替换面很明确；缺点是：理解成本高，很多问题必须同时理解 Cordis、profile、bundle、scope、event 和 service seam。

3.2 OpenCode : 产品入口 + HTTP Server + Session/Tool/Provider 层

OpenCode 的源码目录呈现典型产品化分层：`packages/opencode/src/session` 包含 compaction、message、processor、prompt、retry、revert、run-state 等；`packages/opencode/src/tool` 包含 bash/edit/grep/glob/lsp/code-mode/apply-patch 等；同时还有 cli、console、desktop、server、client、plugin、llm 等包。

OpenCode 文档强调 server 模式：`opencode serve` 启动 headless HTTP server，默认暴露 OpenAPI 3.1、SSE event stream、session/message/tool/LSP/MCP 等端点。这使它很适合作为一个本地或局域网内的 coding agent 服务来集成。

3.3 设计模式对照

设计模式	DeepSeek Harness	OpenCode	本质差异
Plugin Tree	Cordis rows / profiles / bundles / patch overlays	packages + plugins + MCP + config	DSH 更彻底，OpenCode 更产品友好

Capability Seam	Service Definition + Provider + Consumer	provider/model/tool/MCP/Server API	DSH 把 seam 当架构中心
Event Sourcing	SessionEvent append-only log	Session/message API + event stream	DSH 更接近事件源系统
Policy Pipeline	ToolRuntime waterfall + monotonic guard	Permission ruleset allow/deny/ask	DSH 更强治理，OpenCode 更易配置
Client-Server	Web app 是插件层之一	Server 是核心产品能力之一	OpenCode 多客户端体验更成熟
Programmatic Orchestration	run_code over ToolRuntime	execute over connected MCP tools	同名 Code Mode，深度不同

4. Agent Loop 与会话模型

4.1 DeepSeek Harness 的 Turn / Step / Event Log

DSH 架构文档把 step 定义为“一次模型请求加上它请求的工具执行”；turn 是零个或多个 step。其流程是：turn/start, 领取输入, 组装 prompt sections 与 tool schemas, agent/pre-step, step/start, 写 user/message, derive model

history, agent/request, llm/stream, assistant/chunk*, assistant/message, tool/call*, 工具 pipeline, tool/result*, step/end, 直到没有欠下的请求后 turn/end。

关键是：session log 是模型上下文的 source of truth；`deriveMessages()` 只是从 log 投影出模型历史。架构文档还强调“Model-visible means logged”，即任何进入模型请求的内容都必须能从 log 重建。

```
turn/start
  claim input
  assemble prompt + tool schemas
  agent/pre-step
  step/start
  user/message
  deriveMessages()
  agent/request -> llm/stream -> assistant/chunk* -> assistant/message
  tool/call* -> tools/pre-execute -> tools/execute -> tools/post-execute -> tool/result*
  step/end
turn/end
```

4.2 OpenCode 的 Session / Message / Processor

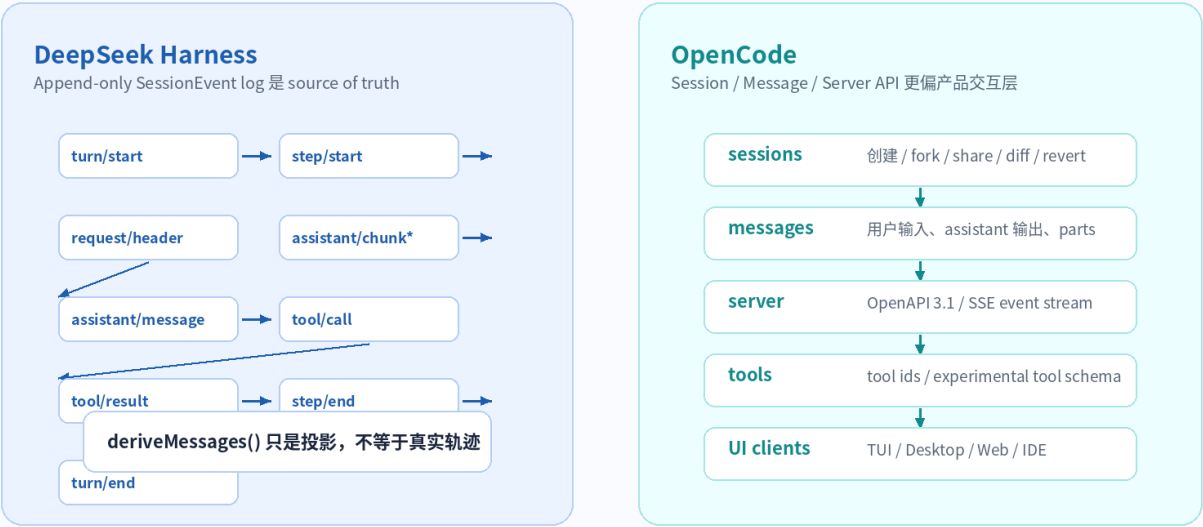
OpenCode 更面向产品使用，其 server 文档直接暴露 Session 和 Message API：创建 session、fork session、share、diff、revert、读取 message、发送 message、获取 shell 等。源码目录中也能看到 `session/message.ts`、`message-v2.ts`、`processor.ts`、`compaction.ts`、`prompt.ts`、`retry.ts`、`revert.ts` 等模块。

OpenCode 的 LLM 设计文档进一步把“Provider Turn”和“Model Run”区分

开：`generateTurn/streamTurn` 表示一次 provider 请求，不执行本地工具；`generate/stream` 表示完整 model run，可以执行工具并继续直到停止条件。这种分离对 durable agent runtime 很有价值。

4.3 轨迹/训练视角

轨迹与上下文：Event Ledger vs Session API



5. 工具系统对比

工具系统对比：Policy Kernel vs Permission Rules

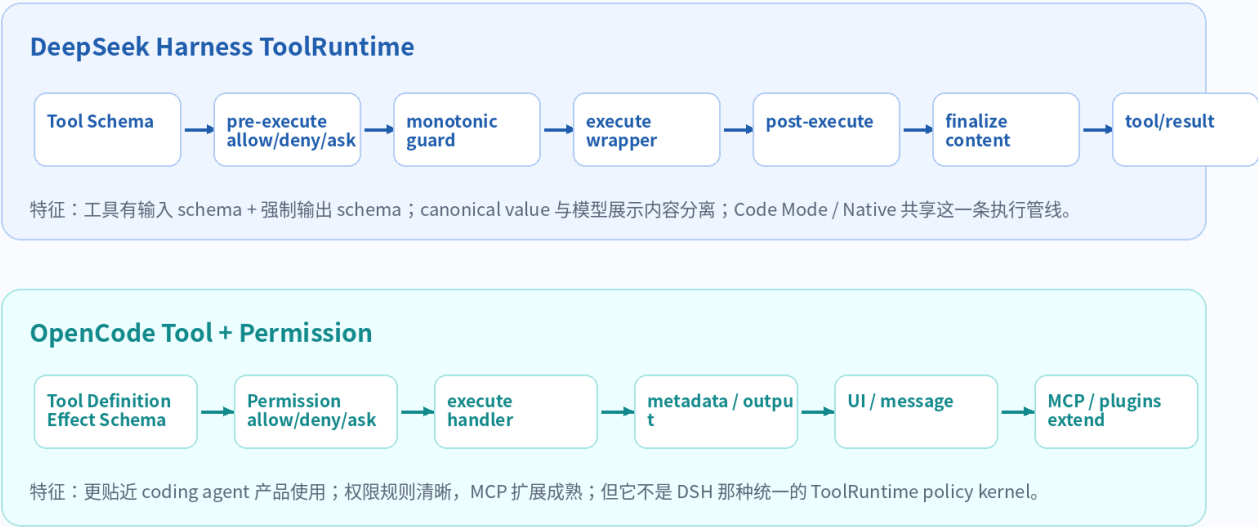


图 4：工具执行管线与治理方式对比

5.1 DSH ToolRuntime：统一工具内核

DSH 的 ToolRuntime 文档给出了非常明确的 pipeline：工具插件注册 schemas 和 executors；每次调用经过 `tools/pre-execute` allow/deny/ask、monotonic guards、`tools/execute` around-dispatch wrapper、`tools/post-execute`、definition-owned `finalizeContent`，最后触发 observe-only `tools/result`。这条 pipeline 的重要性在于：Native Tool Calling 和 Code Mode 都复用它。也就是说，PTC 并没有绕过权限、guard、审计和 schema 校验。

DSH 的 ToolDefinition 还要求强制的 canonical output declaration：工具 body 返回 canonical JSON value，注册时声明 output schema 和 render。模型看到的是 rendered content，程序化消费者拿到的是 canonical value。这一点对公司内部工具平台很重要。

5.2 OpenCode Tool 与 Permission

OpenCode 的工具文档更贴近日常使用：bash 执行 shell 命令，edit/write/apply_patch 修改文件，glob/grep 查找，LSP 提供语言能力，MCP 扩展外部工具。默认工具行为由权限系统控制，可以配置 allow/deny/ask，也可以按工具、按路径或命令 pattern 配置。

源码中的 permission service 使用 wildcard 规则进行 evaluate，默认找最后一个匹配规则；deny 直接失败，allow 通过，ask 则进入 pending 请求。用户可以 once/always/reject。`visibleTools()` 还会根据规则隐藏被 deny 的工具。

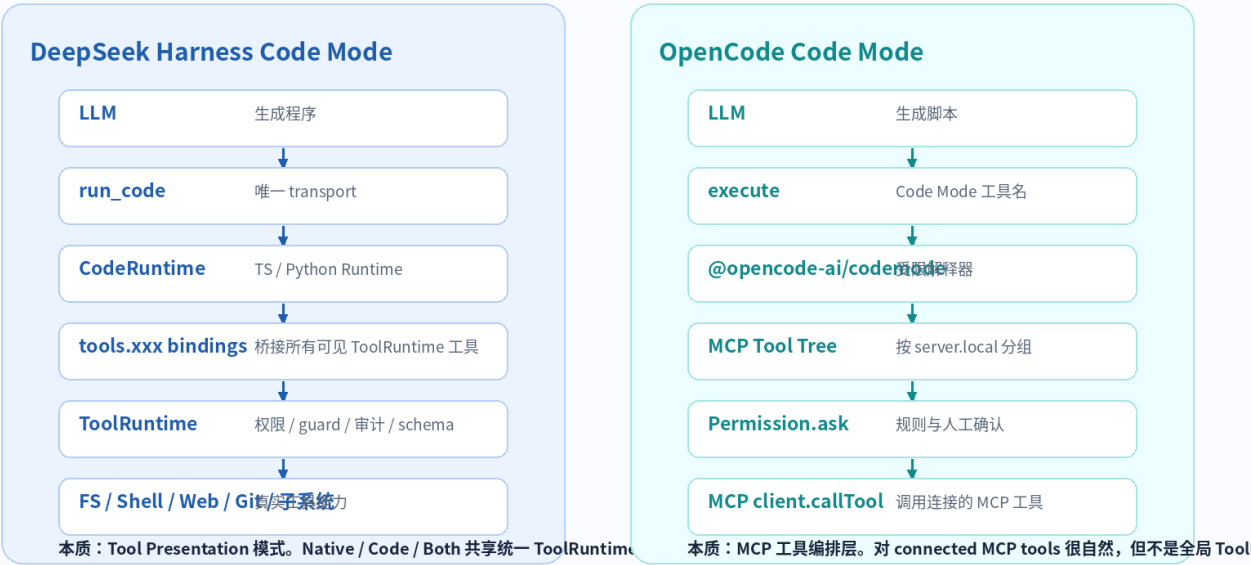
5.3 两者最关键区别

能力	DeepSeek Harness	OpenCode	解释
工具契约	输入 schema + 强制输出 schema + render	Effect Schema / 工具输出 / metadata	DSH 更像 Tool Contract Platform
权限管线	pre-execute + guard + execute	ruleset allow/deny/ask +	DSH 更系统，OpenCode 更直接

	+ post + finalize	UI/user reply	
并发分类	`isConcurrencySafe(args)` 决定 parallel/exclusive	LLM 设计文档中强调同 turn 独立工具可并发	DSH 的 ordered commit 与 exclusive barrier 更细
UI 展示	Tool-owned presentation intents, 不需 UI 按 tool name special-case	工具输出和 metadata 支撑 UI	DSH 抽象更强, OpenCode 更产品实现
扩展路径	Cordis plugin register tool	custom tools / MCP / plugins	OpenCode 接入门槛更低

6. PTC / Code Mode 深度比较

PTC / Code Mode 对比：同名概念，不同集成深度



判断：DSH 的 PTC 是运行时级模式；OpenCode 的 Code Mode 更像产品里的 MCP orchestration capability。

图 5：两者 Code Mode 的集成边界不同

6.1 DeepSeek Harness Code Mode

DSH 的 `tools.mode` 有 native / code / both。native 将可见工具作为 function definitions 提供给模型；code 则贡献保留的 `run\_code` transport、生成的 `tools:sdk` section，以及 code-only rule；both 同时提供两种形式。每个 agent 还能通过 `presentAs(mode)` 覆盖默认 presentation。

因此，DSH 的 Code Mode 不是第二套工具系统，而是同一个 ToolRuntime 的另一种模型侧 presentation。内部工具能力不变，模型看到的接口从多个 JSON function collapse 成 `run\_code + generated SDK`。

其源码 `createRunCodeTool` 的 scheduler 还强调：start 按 submission order，body 可并发，result commit 按 submission order；parallel 调用可重叠，exclusive 调用等待 pool drain，并直到 post-execute/commit 完成才释放 barrier。这是 deterministic concurrency，而不只是 Promise.all。

6.2 OpenCode Code Mode

OpenCode 也有 `packages/opencode/src/tool/code-mode.ts` 和 `packages/codemode`。当前源码显示其模型侧工具名是 `execute`，描述为“Run a confined orchestration script with access to connected MCP tools”。它会读取当前 agent/session 权限，筛选可见 MCP tools，按 server/local 分组成工具树，然后通过 `@opencode-ai/codemode` 运行脚本。

OpenCode Code Mode 每次 child tool call 会先触发 plugin hook，再 `Permission.ask`，然后通过 MCP client `callTool` 调用对应 MCP 工具；运行中通过 metadata 发布 toolCalls 的 running/completed/error 状态；最终把 result.value stringify 后作为输出，并收集附件。

6.3 PTC 设计差异

比较点	DeepSeek Harness	OpenCode
模型侧工具名	run_code	execute

工具来源	当前 agent 可见的 ToolRuntime 工具	connected MCP tools
架构身份	Tool presentation mode	MCP orchestration tool
权限/审计	完整复用 ToolRuntime pipeline	Permission.ask + plugin hooks + MCP call
类型来源	Tool schema -> generated TS/Python SDK	MCP input/output schema -> codemode tool tree
中间调用轨迹	inner dispatch log-only, 外层输出进入模型	metadata toolCalls 追踪 running/completed/error
适合场景	内部工具平台、批量读写、跨系统检索、长任务压缩上下文	MCP 生态工具批量编排、外部服务聚合

结论：两者都在把“模型一步一步调用工具”转变为“模型写程序编排工具”。但 DSH 把它做成 ToolRuntime 的一等 presentation；OpenCode 当前实现更像一个连接 MCP tools 的产品功能。这不是优劣绝对判断，而是集成深度和架构目标不同。

7. Schema 与类型契约

7.1 DSH 的 Schema 更像 Agent IDL

DSH 的工具 schema 不是只服务于 function calling。它同时用于参数校验、输出校验、Native tool schema、Code Mode SDK 生成、Subagent/Workflow structured output、UI presentation 等。ToolDefinition 要求 mandatory output declaration，使工具结果可以同时服务于模型、程序化执行和 UI。DSH 文档明确说明其统一 schema DSL 支持 string、number、integer、boolean、null、array、object、json、exact-one oneOf；显式 object 必须声明 additionalProperties；深 schema 的编译、验证、TypeScript 渲染使用 explicit work stack，避免调用栈爆炸。

7.2 OpenCode 的 Schema 更贴近 Effect / Product API

OpenCode 工具源码大量使用 Effect Schema，例如 Code Mode 的 Parameters 是 `Schema.Struct({ code: Schema.String })`。OpenCode server 还通过 OpenAPI 3.1 暴露 HTTP API，`/experimental/tool` 可按 provider/model 返回工具定义。OpenCode 的 LLM 设计文档也强调 portable request data 与 local executable behavior 分离：request、messages、tool definitions、events、usage 等是 plain immutable data；configured models、executable tools、hooks 和 provider definitions 可包含函数和 Effect requirements，不能直接序列化。

维度	DeepSeek Harness	OpenCode
Schema 角色	跨 Tool/PTC/Subagent/Workflow/UI 的统一类型契约	工具参数、API、MCP、Effect runtime 类型
输出 schema	强制 canonical output schema + render	工具输出结构依工具而定，MCP/Effect Schema 支撑
PTC SDK	由 Tool Schema 生成 TS/Python SDK	由 MCP tool catalog 构造 codemode tool tree
失败策略	unsupported schema fail loud, renderer 可 conservative fallback	typed errors / permission errors / Effect error path
企业启发	建立公司级 Tool Contract Platform	建立易接入的产品 API 与 MCP 工具生态

8. 安全模型与权限治理

8.1 DSH : Sandbox 是文件 effect policy seam

DSH 的 sandbox 文档明确说明：process sandbox seam 将 same-world subprocess argv 包装在 file-effect policy 里；local provider 在 Linux 使用 bwrap/Landlock，在 macOS 使用 Seatbelt，在 Windows 使用 ACL restricted-token backend。

SandboxMode 包括 read-only、workspace-write、danger-full-access；它只管理 filesystem effects，不管理网络或进程可见性。danger-full-access 直接绕过 confinement；read-only/workspace-write 必须有 backend。官方明确要求 silent unconfined passthrough never legal，也就是 fail-closed。

8.2 OpenCode : Permission Rules 更贴近产品交互

OpenCode 权限文档支持全局和工具级 allow/deny/ask，bash 可以按命令 pattern 控制，edit/write/apply_patch 共享 edit 权限，read 可按路径模式配置。源码中的 Permission.evaluate 使用 Wildcard 匹配并选择最后一个匹配规则；reply 支持 reject、once、always，并能更新 session 内已批准规则。

这套机制非常适合用户交互和团队配置，但它本质上不是 DSH sandbox 那样的 OS/filesystem confinement。换句话说，OpenCode 的 permission 是“是否允许 agent 做某事”，DSH 的 sandbox 是“即使允许了，也在 host 层限制文件 effects”。

安全问题	DeepSeek Harness	OpenCode	企业建议
误执行高危命令	通过 approval + sandbox mode + backend fail-closed	通过 permission ask/deny/pattern	两者都需要；permission 管入口，sandbox 管后果
文件写入边界	workspace-write 只允许 workspace/temp，需 backend	edit 权限控制写入，非 OS sandbox	高安全场景要补容器/沙箱
长期运行/后台任务	jobs/subprocess/sandbox seam	background/session/tool 状态	需要统一 job 审计
网络/进程隔离	DSH 文档明确不覆盖网络/进程可见性	权限也不等价网络隔离	生产要加 Docker/microVM/network policy

9. 使用体验对比

9.1 OpenCode 的优势：用户路径短

OpenCode 对普通开发者更友好。官方 README 一屏内给出 curl 安装、npm、brew、scoop、choco、nix、Arch、mise 等路径；还提供 Desktop App beta。内置 build/plan/general agents 的概念也更容易理解：build 负责开发工作，plan 只读分析，general 子代理处理复杂搜索和多步任务。

OpenCode server 模式让它更适合集成：默认端口 4096，可通过 OpenAPI/SSE 访问 session、message、tool、LSP、MCP 等资源；TUI 启动时本身也会启动 server，支持多客户端架构。

9.2 DSH 的优势：架构可解释，平台可塑性强

DSH 目前的产品体验不如 OpenCode 直接，但它的架构文档、subsystem 文档、generated catalog、pipeline 图、persistence catalog、tool catalog 等非常适合工程团队研究。它像一个“可研究、可替换、可治理”的 runtime kernel。

尤其在公司内部平台里，经常要回答：工具如何审计？工具输出如何类型化？模型看见的内容和真实执行日志如何区分？sandbox 失败时是否允许裸跑？PTC 如何不绕过权限？这些问题 DSH 给出了更系统的设计。

9.3 使用体验评分

维度	DeepSeek Harness	OpenCode	说明
安装与首次启动	3/5	5/5	DSH 可 npm/source；OpenCode 安装路径更多、用户文档更成熟
TUI/CLI 体验	2.5/5	5/5	OpenCode 以 TUI/编码体验为核心卖点
Web/Desktop/Server	3/5	4.5/5	DSH 有 Web UI；OpenCode 有 desktop beta 和 headless server
配置可理解性	2.5/5	4/5	DSH profile/bundle/patch 强但复杂；OpenCode config 更直观
源码可研究价值	5/5	4/5	DSH 架构抽象更浓；OpenCode 产品工程更完整
企业治理潜力	5/5	3.5/5	DSH ToolRuntime/sandbox/event ledger 更适合治理内核

10. 企业内部落地：怎么借鉴两者

选型矩阵：不要问谁“更强”，要问你要什么能力

场景	更推荐	理由
日常编码体验 / 团队采用	OpenCode	安装、TUI/桌面/服务端、MCP/模型生态更成熟
内部 Agent 平台内核	DeepSeek Harness	插件树、capability seam、event ledger、sandbox 策略更系统
复杂工具治理 / 审计	DeepSeek Harness	ToolRuntime pipeline、强制输出 schema、monotonic guard
快速集成 MCP 与多模型	OpenCode	MCP 配置、provider 生态、OpenAPI/SDK 更顺手
PTC / 程序化工具编排	DeepSeek Harness	Code Mode 是 ToolRuntime presentation；OpenCode 主要编排 MCP tools
训练轨迹 / 可回放研究	DeepSeek Harness	append-only SessionEvent 更接近 Agent execution ledger
稳态产品可用性	OpenCode	DSH 仍是 developer preview，OpenCode 更面向用户可用
安全边界思想	DeepSeek Harness	workspace-write fail-closed、平台 sandbox backend 明确

总体建议：短期业务落地以 OpenCode 为产品入口；中长期平台建设吸收 DeepSeek Harness 的内核设计。

图 6: 企业选型矩阵

10.1 推架：OpenCode 做入口，DSH 思想做内核

如果公司内部要建设 AI coding / agent 平台，不建议只复制某一个项目。更稳妥的方式是：OpenCode 思路解决用户入口和多模型/MCP 生态，DeepSeek Harness 思路解决工具内核、轨迹、schema、sandbox、PTC 和权限治理。

- User / Developer
- > TUI / Desktop / Web / IDE 入口（OpenCode 式体验）
 - > Internal Agent Gateway
 - > Tool Contract Platform（DSH 式 input/output schema + canonical value）
 - > ToolRuntime Policy Kernel（approval / guard / audit / timeout / metrics）
 - > PTC Runtime（run_code / workflow / batch orchestration）
 - > Execution Layer（sandbox / container / remote executor）
 - > Event Ledger（raw events + model view + execution view）

10.2 五阶段落地路线

阶段	目标	借鉴对象	关键产物
Phase 1	统一开发者入口	OpenCode	安装包、TUI/Server、Provider 配置、MCP 接入
Phase 2	统一工具契约	DeepSeek Harness	Tool Schema IDL、input/output schema、canonical output、renderers
Phase 3	统一权限与审计	DeepSeek Harness + OpenCode	permission rules + approval UI + ToolRuntime audit events
Phase 4	引入 PTC	DeepSeek Harness	run_code sandbox、

			generated SDK、parallel/exclusive、ordered commit
Phase 5	训练与评测闭环	DeepSeek Harness	raw event ledger、model view、execution view、reward/eval pipeline

10.3 公司级 Tool Contract 建议

建议不要让每个工具只返回一段 Markdown。公司内部工具应该采用类似 DSH 的强契约：输入 schema、输出 schema、execute、render for LLM、render for UI、permission class、concurrency class、audit metadata。

```
tool: gitlab.search_merge_requests
input:
  project: string required
  state: enum(opened, merged, closed)
output:
  items: array<object>
  total: integer
policy:
  permission: read
  concurrency: parallel
render:
  llm: summarized text
  ui: search card
audit:
  pii: false
  retention: 180d
```

11. 关键 Gotchas

项目	Gotcha	影响	建议
DeepSeek Harness	developer preview, 破坏性变更明确存在	不宜立即绑定长期生产 API	先抽象设计原则，避免深度耦合未稳定接口
DeepSeek Harness	workspace-write 需要真实 sandbox backend, 否则 fail-closed	本地/容器环境可能频繁报 SANDBOX_UNAVAILABLE	开发环境修 bwrap/Landlock/Seatbelt, 容器内可视情况 danger-full-access
DeepSeek Harness	PTC 中间 canonical value 不一定持久化到 Session Log	训练数据可能缺真实工具返回对象	在企业版 trace 里额外采 canonical input/output
OpenCode	Permission 不是 OS sandbox	允许后仍可能对本机产生真实影响	高风险环境用容器/VM/microVM 包裹
OpenCode	MCP 工具生态强但治理容易发散	工具来源、命名、权限、schema 一致性变复杂	建立公司 MCP/Tool 审核与 schema lint
OpenCode	Code Mode 当前更偏 MCP tool orchestration	不等价 DSH 全 ToolRuntime presentation	不要把两者 PTC 能力简单划等号
两者共同	模型能力决定工具编排质量	弱模型写错参数/错误循环/过度调用工具	需要 eval、限流、超时、重试、轨迹分析

12. 综合判断

OpenCode 的强项是“让开发者用起来”。它的 TUI、Desktop、Server、SDK、MCP、provider 生态、内置 agents 与 permission 配置，使它更像一个真正可以日常使用的 coding agent 产品。

DeepSeek Harness 的强项是“让平台工程师看懂下一代 Agent Runtime 该怎么做”。它把 ToolRuntime、SessionEvent、System Prompt Assembly、Sandbox、PTC、Capability Seam 和 Plugin Tree 放在统一框架里，适合公司内部长期平台建设。

因此，本报告的最终建议是：短期如果要部署一个 AI coding assistant，可以优先试 OpenCode；中长期如果要建设公司内部 AI Harness，尤其是你关心的 DeepSeek V4 Flash、本地模型、Skill Hub、PTC、Agent Loop 与审计数据，那么 DeepSeek Harness 的设计原则更值得系统吸收。

12.1 最值得复制的三个 DeepSeek Harness 思想

4. Tool Capability 与 Tool Presentation 分离：同一能力可以以 native/code/both 等不同形式暴露给模型。
5. Event Ledger 与 LLM Context 分离：原始执行轨迹不等于模型上下文，后者只是前者投影。
6. ToolRuntime 作为 Policy Kernel：权限、guard、schema、审计、并发、结果渲染走统一管线。

12.2 最值得复制的三个 OpenCode 思想

7. 以开发者体验为先：TUI/桌面/server/API/多模型/插件生态同时推进。
8. MCP 和 permission 配置产品化：用 allow/deny/ask 和 pattern 让用户可理解地控制工具。
9. Server-first integration：OpenAPI/SSE/session/message/tool/LSP/MCP 端点让外部系统容易接入。

附录 A：源码与文档索引

以下为本报告引用和核对过的主要官方资料。为避免将工具内部 citation token 泄漏进交付文档，这里使用普通人类可读的仓库路径索引。

编号	项目	路径/文档	用途
D1	DeepSeek Harness	deepseek-ai/deepseek-harness / README.md	项目定位、developer preview、运行方式
D2	DeepSeek Harness	docs/architecture.md	Cordis、profiles/bundles、core packages、turn flow、session log、capability seams
D3	DeepSeek Harness	packages/core/tools/ README.md	ToolRuntime pipeline、tools.mode、Code Mode、schema、guard、output contract
D4	DeepSeek Harness	packages/core/tools/src/code-mode.ts	createRunCodeTool、bindings、scheduler、ordered commit、parallel/exclusive
D5	DeepSeek Harness	docs/subsystems/sandbox.md	SandboxMode、bwrap/ Landlock/Seatbelt/Windows ACL、fail-closed
D6	DeepSeek Harness	packages/code-runtime/code-runtime-worker-thread/ README.md	fresh worker、containment not security boundary、resource caps
O1	OpenCode	anomalyco/opencode / README.md	项目定位、安装方式、Desktop beta、built-in agents
O2	OpenCode	opencode.ai/docs/tools	内置工具、bash/edit/write、MCP/custom tools
O3	OpenCode	opencode.ai/docs/permissions	allow/deny/ask、bash pattern、edit/read permissions、per-agent
O4	OpenCode	opencode.ai/docs/server	headless server、OpenAPI 3.1、SSE、session/message/tool/LSP /MCP endpoints
O5	OpenCode	opencode.ai/docs/agents	build、plan、general、explore、scout、compaction/title/summary agents
O6	OpenCode	opencode.ai/docs/mcp	local/remote MCP、OAuth、tool filtering、per-agent MCP
O7	OpenCode	packages/opencode/src/tool/ code-mode.ts	execute tool、MCP tool tree、Permission.ask、runtime.execute、metadata
O8	OpenCode	packages/opencode/src/ permission/index.ts	Permission.evaluate/ask/reply/ visibleTools
O9	OpenCode	packages/llm/DESIGN.md	Provider Turn vs Model Run、Tool concurrency、portable request data
O10	OpenCode	packages/opencode/src/ session/* 与 packages/opencode/src/tool/*	源码结构：session/message/ processor/prompt/tools

附录 B：术语速查

术语	解释
Agent Harness	围绕模型、工具、会话、权限、上下文、执行环境构建的运行时框架。
Capability Seam	服务定义、服务提供者、服务消费者组成的可替换能力边界。
ToolRuntime	工具注册、schema、权限、执行、审计和结果渲染的统一管线。
PTC / Code Mode	模型先生成程序，再由程序批量/并发/条件地调用工具。
Event Ledger	append-only 的真实执行事件账本，可从中投影模型上下文、UI 展示和审计视图。
Canonical Value	工具真实结构化返回值，区别于给模型看的文本展示。
Permission	是否允许某个操作发生的决策层。
Sandbox	对进程文件 effects 做系统级限制的执行边界，不等同于 permission。
MCP	Model Context Protocol，用于连接外部工具、资源和提示能力。